FACULDADE DE TECNOLOGIA DE SINOP

Compiladores

ENGENHARIA DE COMPUTAÇÃO
FASTECH SINOP

Prof. Ramilton Costa Gomes Júnior

FACULDADE DE TECNOLOGIA DE SINOP

ENGENHARIA DE COMPUTAÇÃO FASTECH SINOP

Agenda

- Análise léxica;
- Conceitos e Padrões de Tokens ;
- Definição de lexemas e tokens;
- Uso de Expressões Regulares

Análise Léxica

Análise léxica

- **O que é análise léxica?**

É uma etapa da análise de um computador que divide o código fonte em tokens.

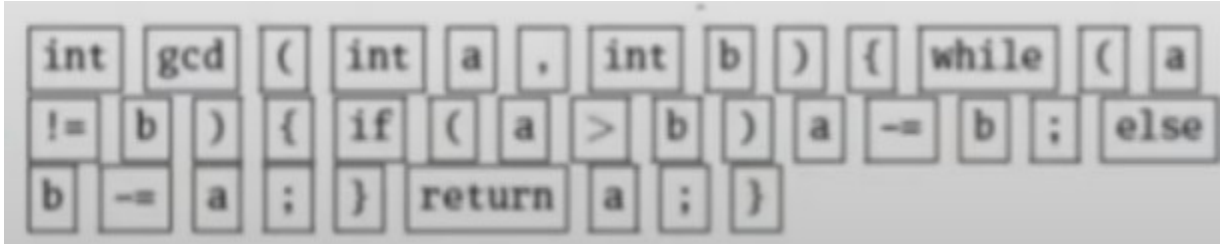
- Análise léxica

- Recebe uma seqüência de caracteres e produz uma seqüência de palavras chaves, pontuação e nomes;
- Descarta comentários e espaços em branco;
- Organiza caracteres de entrada em grupos, chamados tokens;

- Análise léxica

- Também chamado de scanner
- Tarefa principal
  - Ler o arquivo ond ese encontra o programa fonte;
  - Agrupa em sequência o que chamamos de lexemas;

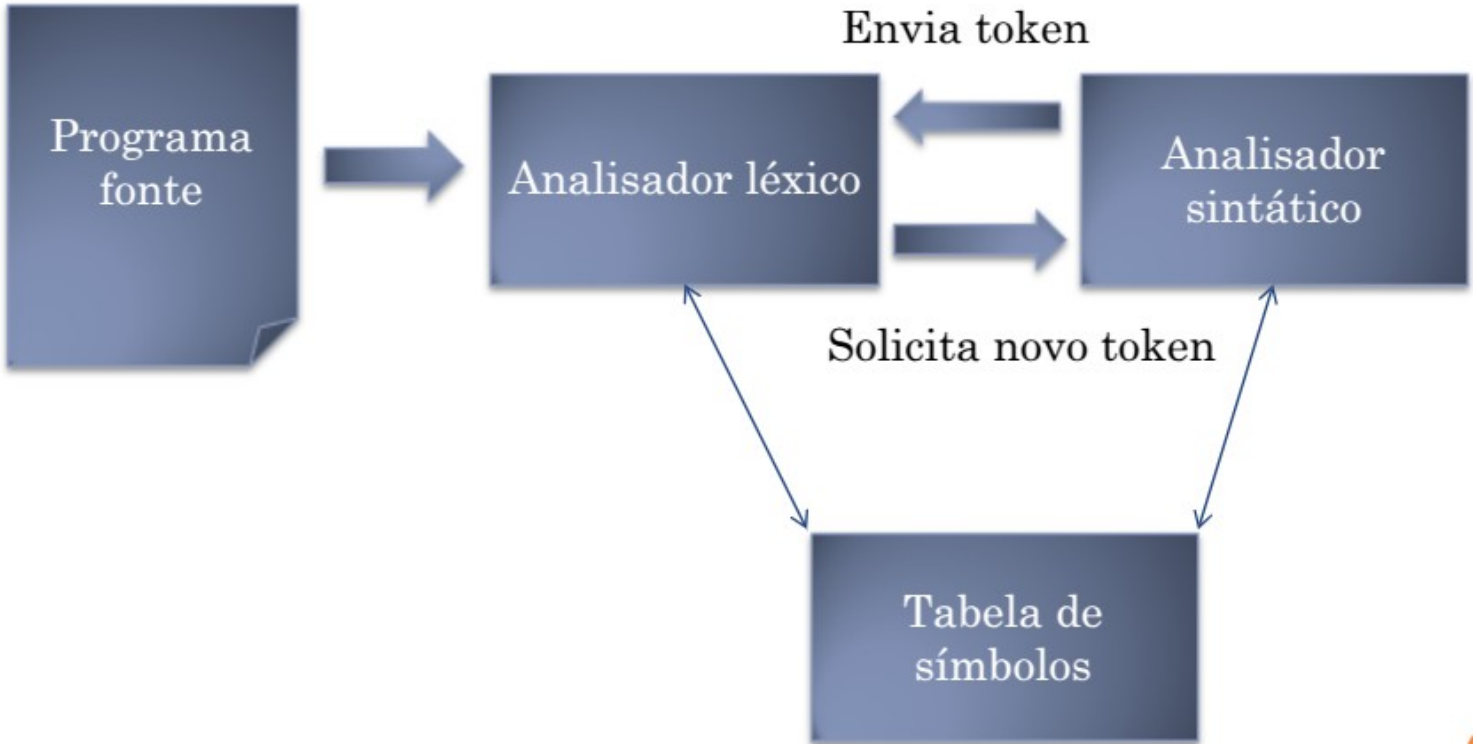
```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```



- **Análise léxica**
  - É a parte do compilador responsável por ler caracteres do programa fonte e transformá-lo em uma representação conveniente para o analisador sintático.
  - O analisador léxico lê o programa fonte caracter a caracter, agrupando os caracteres lidos para formar os símbolos básicos (tokens) da linguagem;
    - Identificadores, palavras-chaves, operadores, parênteses e sinais de pontuação.

- **Análise léxica**
  - Vantatens da divisão em análise léxica e sintática:
    - Projetos mais simples. Diminui a complexidade do analisador sintático que não precisa mais lidar com estrutura foras de seu escopo como tratamento de caracteres vazios;
    - Melhora a eficiência do compilador. Técnicas de otimização específicas para o analisador léxico.
    - Melhora a portabilidade.

- **Cenário**


Programa fonte
Analisador léxico
Envia token
Analisador sintático
Solicita novo token
Tabela de símbolos

Conceitos e padrões de tokens

- O que são tokens?

São unidades significativas de um programa que são reconhecidas pelo compilador. São palavras e sinais de pontuação que expressão a estrutura do programa.

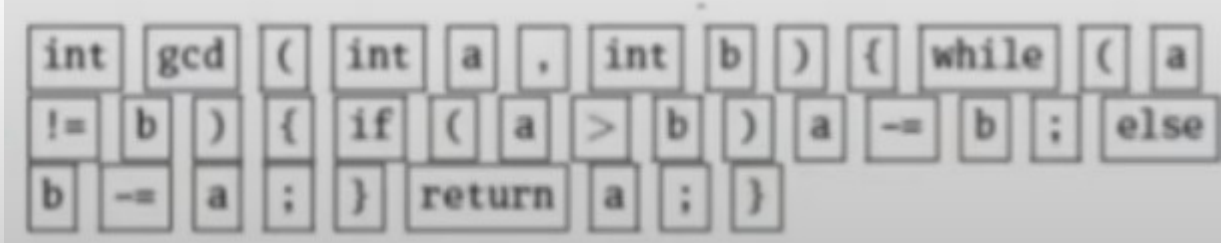
Como são os tokens?

- São formados por um ou mais caracteres, sem incluir espaços em branco, caracteres de controle, ou caracteres dentro de um identificador;
- São o menor elemento de um programa que é significativo para o compilador;
- São formados por um nome de token e um valor de atributo opcional;

Como são os tokens?

- O nome do token é um símbolo abstrato que representa um tipo de unidade léxica;

```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```



- Tipos de tokens

- Palavras-chave
- Identificadores
- Literais de ponteiro, numéricos e boolianos
- Literais de cadeia de caracteres e de caracteres
- Literais definidos pelo usuário
- Operadores e Pontuadores;

- Tipos de tokens

- Identificadores
  - `nome_de_variavel`
  - `nome_de_classe`
- Palavras reservadas
  - `if, while, class, for...`
- Números inteiros sem sinal
  - `10, 234, 0`
- Números reais
  - `1.8, 2.9, 0.1`
- Caracteres especiais
  - `$, º, ª`
- Comentários
  - `/* isto é um comentário */`

- Reconhecimento de tokens

- If -> if
- Then -> then
- Else -> else
- Relop -> <|<=|=|<>|>|>=|
- Id -> letra(letra|digito)*
- Num -> digito+ (.digito+)?(E(+|-)? digito+)?

Definição de lexamas e tokens

- Definição

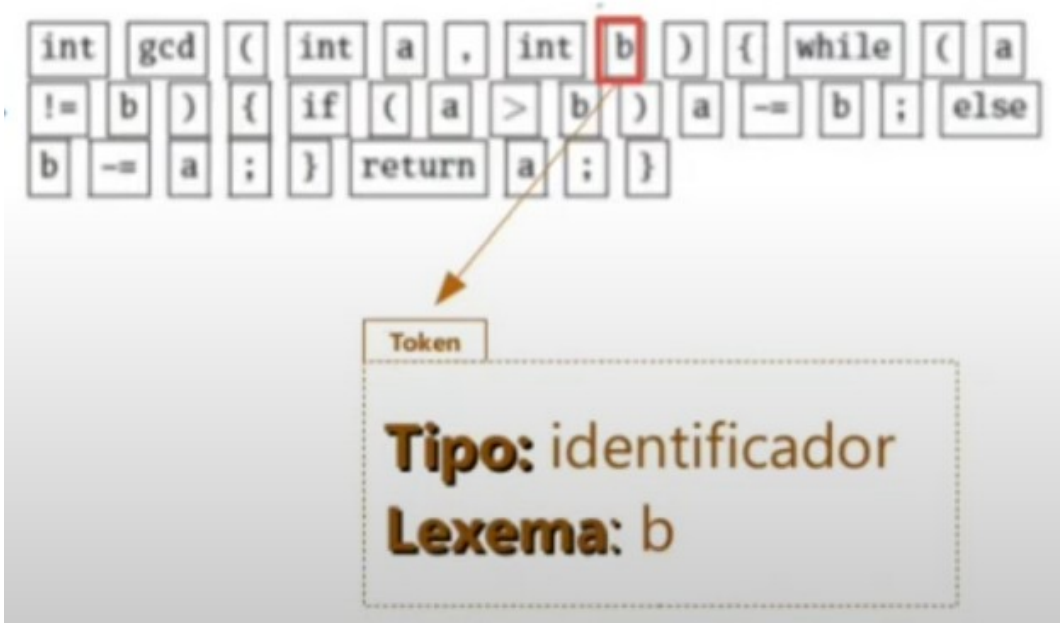
- Lexemes
  - Sequência de caracteres que forma uma unidade léxica;
- Tokens:
  - Um objeto que contém um par de informações
    - Identificador do tipo da unidade léxica;
    - E um valor;

- **Exemplo**

```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```

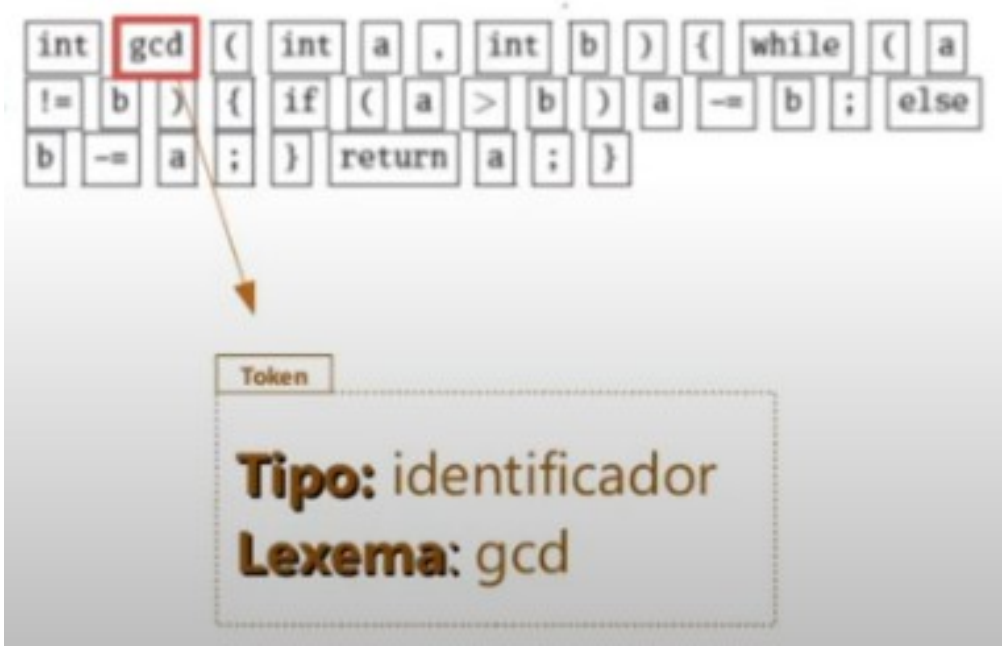
- **Exemplo**

```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```



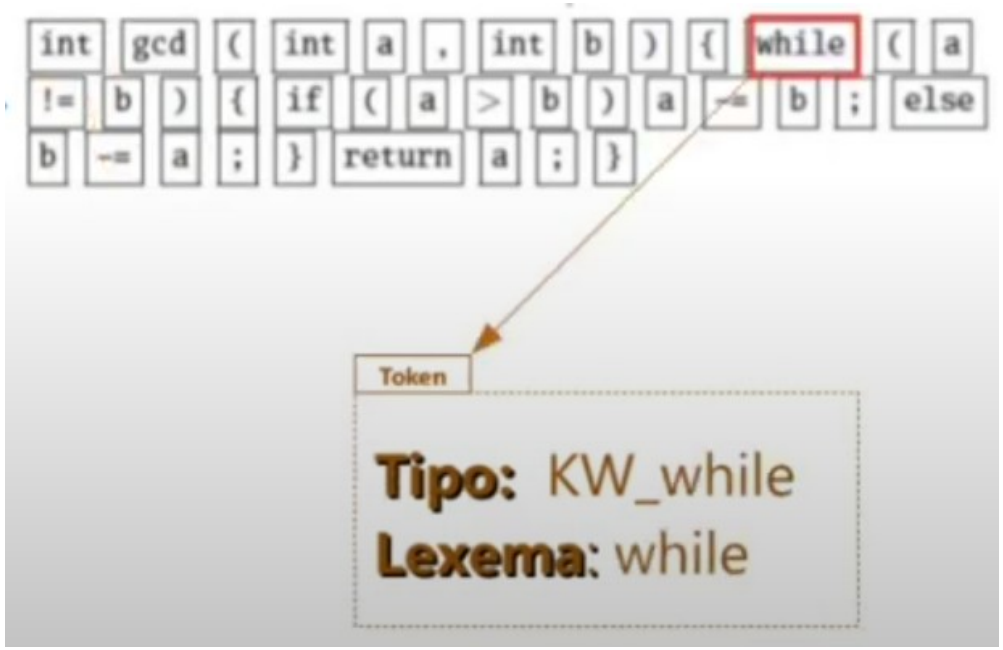
- **Exemplo**

```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```



- **Exemplo**

```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```



- Análise léxica tokens

- Ao ler um identificador:
  - Nome de variável
  - Nome de função
  - Nome de classe
- Gerar o seguinte token:
  - Tempo -> <IDENT, tempo>
  - Soma -> <IDENT, soma>

- Análise léxica tokens

- Ao ler uma palavra chave:
  - If, while, for
- Gerar o seguinte token:
  - if -> <KW_IF, “if”>
  - while -> <KW_WHILE, “while”>
  - class -> <KW_CLASS, “class”>
  - (-> <LEFTPAR, “(”>

- Análise léxica

- Cada linguagem tem seu conjunto de tokens que sao aceitos;
  - Pascal: existe a palavra-reservada then;
  - C não possui;
- A principal função do analisador léxico é:
  - Ler o arquivo
  - Verificar se o que foi lido é um token válido.

- Análise léxica

- Ao conjunto de tokens aceitos dá-se o nome:
  - Tabela de simbolos;
- Funciona como uma base de dados para consulta;
  - Útil para todas as fases seguintes do compilador;
- Também utilizada para armazenar
  - O tipo das variáveis;
  - O valor das variáveis e seu escopo.

- Tabela de simbolos

- Seja o código:

```
fi ( a < 10 ) {
    return a;
}
```

- Inicia a leitura do texto
- Captura as letras f e dpois i
- O analisador léxico verifica na tabela de simbolos se:
  - Fi é uma palavra reservada?

- Tabela de simbolos

- Seja o código:

```
fi ( a < 10 ) {
    return a;
}
```

- Como não é encotrada na tabela de símbolos, é criado um identificador.
  - <IDENT, FI>

- Tabela de simbolos

- Exemplo
  - Soma = a + b *10;

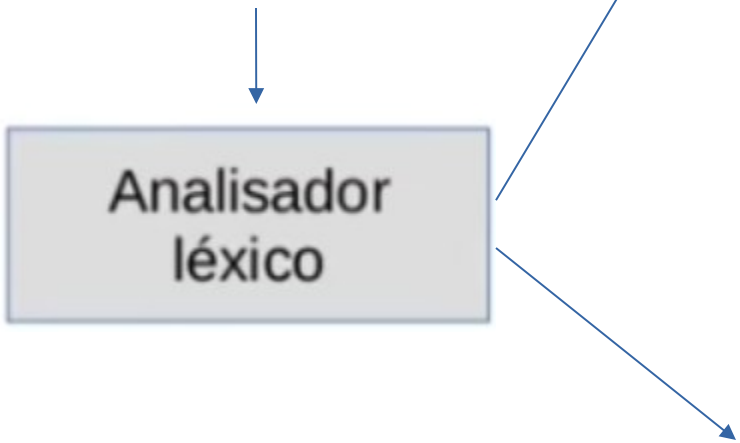
Analisador léxico

<IDENT, apontador p/ posição soma na tabela de simbolos>,
<OP_IGUAL>,
<IDENT, apontador p/ posição a na tabela de simbolos>,
<OP_SOMA>,
<IDENT, apontador p/ posição b na tabela de simbolos>,
<OP_MULT>,
<NUM, valor 40>

Tabela de simbolos			
	Lexema (lido)	token	valor
1	soma	<IDENT>	-
2	=	<OP_IGUAL>	-
3	a	<IDENT>	-
4	+	<OP_SOMA>	-
5	b	<IDENT>	-
6	*	<OP_MULT>	-
7	40	<NUM>	40

- Tabela de simbolos

- Exemplo
  - Soma = a + b *10;



<IDENT, 1>, <OP_IGUAL>,
<IDENT, 3>, <OP_SOMA>,
<IDENT, 5>, <OP_MULT>,
<NUM, valor 40>

Tabela de simbolos			
	Lexema (lido)	token	valor
1	soma	<IDENT>	-
2	=	<OP_IGUAL>	-
3	a	<IDENT>	-
4	+	<OP_SOMA>	-
5	b	<IDENT>	-
6	*	<OP_MULT>	-
7	40	<NUM>	40

- **Atividade**

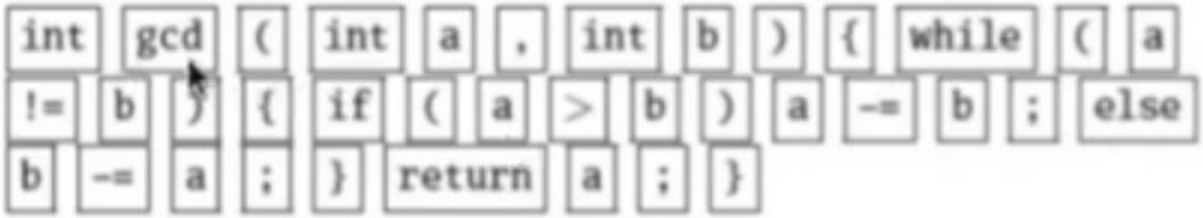
```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```

```
i n t sp g c d sp ( i n t sp a sp , sp i n t sp b
) { nl w h i l e (a ! = b ) nl { i f sp ( a > b ) nl
a - = b ; nl e l s e nl b - = a ; nl } nl
r e t u r n sp a ; nl }
```

- O analisador léxico, vê o codigo fonte como uma sequência de caracteres;
- O analisador léxico unirá os caracteres de acordo com os tokens da linguagem;

- Atividade

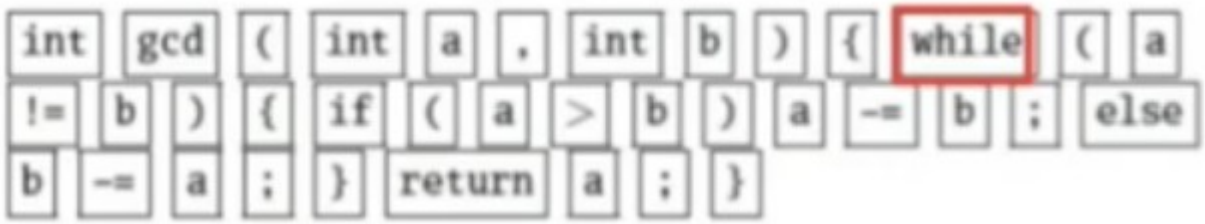
```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```



- Note que, o analisador léxico ignora:
  - Espaços em branco
  - Comentáios, new line, etc.

- **Atividade**

```
int gcd (int a, int b){
    while(a != b){
        if(a > b)
            a -= b;
        else
            b -= a;
    }
    return a;
}
```



- Cada token tem um código/nome
- While -> <KW_WHILE>

- **Atividade**

- X = 60.8 + id2;
- Flag = x1 * 2;
- If (a = b) then

  Write(“Analise léxica”);
- while(a < b){

  A++;

}

- Formalismo

- Expressões regulares
  - Letra -> [A-Z][a-z]
  - Digito -> [0 – 9]
  - Digitos -> digito digito*
  - Ident -> letra(letra | digito)*

- Autômato Finito

- Formalismo

- Este autômato faz o reconhecimento de identificadores;

```
int soma(int valor1, int valor2){
    int resp = valor1 + valor2;
    return resp;
}
```

Expressão Regular

letra → [A – Z] [a – z]
digito → [0 – 9]
digitos → digito digito*
ident → letra (letra | digito)*

Autômato Finito

- Definições regulares de tokens

- Letra -> [A-Z] [a-z]
- Digito -> [0-9]
- Digitos -> digito digito*
- Ident -> letra(letra | digito)*
- Fração -> .digito | ε
- Exponencial -> E(+|-|ε) digitos | ε
- Num-> digitos fração exponencial

- Definições regulares de tokens

- KW_INT -> int
- KW_WHILE -> while
- Operadores
  - * Zero ou mais ocorrências
  - + Uma ou mais ocorrências
  - ? Zero ou uma ocorrência
  - [...] Classes de caracteres

- Definições regulares de tokens

* Zero ou mais ocorrências
  - + Uma ou mais ocorrências
  - ? Zero ou uma ocorrência
  - [...] Classes de caracteres
    - [a-z]
  - A | B ocorrência de A ou de B
  - AB ocorrência de A seguida ocorrência B

- Diagrama de transições

- Utilizado para determinar a sequência de ações executadas para o analisador léxico no processo de reconhecimento de token;
- As posições no diagrama são representadas através de um círculo e são chamados de estados;
- Os estados são conectados por setas, denominadas lados;

- Diagrama de transições

- Os lados são rotulados com caracteres que indicam as possíveis entrada que podem aparecer após o diagrama de estado ter atingido um dado estado;
- O rótulo outro se refere a qualquer caracter de entrada que não seja o indicado pelos demais rótulos que deixam o estado;
- Um circulo duplo determina um estado de aceitação;

Uso de expressões regulares

- Expressões regulares

- Notação especial para a definição de cadeias de uma linguagem;
- Identificador pascall
  - letras(letra |digito)*
  - Caracter | é igual a ou
  - * significa zero ou mais instâncias;

- Expressões regulares

- A justaposição de letras significa concatenação destas
  - Ex:
    - a | b {a,b}
    - (a|b)(a|b) {aa,ab,ba,bb}
    - a* {a,aa,aaa,aaaa,....}
    - (a|b)*

- Expressões regulares

- Se duas expressões regulares denotam a mesma linguagem, dizemos que são equivalentes e representamos r=s.
  - Ex: (a|b) = (b|a);

- Expressões regulares

- Definições regulares
  - Expressões regulares podem ser nomeadas e estes nomes podem ser utilizados para definir novas expressões;
    - Ex:
      - Letra: A|B|..|Z|a|b|..|z|
        - » [A-Za-z_]

- Expressões regulares

- Definições regulares
  - Expressões regulares podem ser nomeadas e estes nomes podem ser utilizados para definir novas expressões;
    - Ex:
      - Digitos: 0|1|2|..|9|

        » [0-9]

- Expressões regulares

- Definições regulares
  - Numero -> digitos(.digitos)?(E[+-]?digitos)?
  - Id -> letra_(letra_|digito)*

- Expressões regulares

x	o caracter 'x'
"x"	o caracter 'x', mesmo que este seja um meta-caracter
.	todos os caracteres excepto '\n' ("newline")
[xyz]	'x', ou 'y', ou 'z'
[x-z]	todos os caracteres entre 'x' e 'z' (classe)
[^x-z]	todos os caracteres excepto os entre 'x' e 'z'
r*	a expressão regular r, zero ou mais vezes
r+	r, uma ou mais vezes
r?	r opcional
r{n}	r *n* vezes
r{n,}	r *n* ou mais vezes
r{n,m}	r *n* a *m* vezes
{name}	a expansão da definição referente a *nome*.

- Expressões regulares

\x	Se x é igual a 'a', 'b', 'f', 'n', 'r', 't', ou 'v', então tem-se a interpretação ANSI-C, caso contrário dá-nos o caracter 'x' (necessário para se poder aceder aos meta-caracteres).
\0	o caracter nulo (NUL, código ASCII 0).
\123	o caracter com valor octal 123.
\x2a	o caracter com valor hexadecimal 2a
(r)	a expressão regular r; necessário para contornar as regras de precedência, ver mais a baixo.
rs	concatenação das expressões regulares r e s
r|s	r ou s.
r/s	r mas só se for seguindo de um s.
^r	r no princípio de uma linha.
r$	r no fim de uma linha.
<s>r	r, mas só numa condição inicial.

- Extensões de Expressões regulares

Expressão	Casa com	Exemplo
\c	O caractere c literalmente	*
"s"	A cadeia s literalmente	"**"
.	Qualquer caractere menos quebra de linha	a.*b
^	O início de uma linha	^abc
$	O fim de uma linha	abc$
[s]	Qualquer um dos caracteres na cadeia s	[abc]
[^s]	Qualquer caractere não presente na cadeia s	[^abc]

Expressões Regulares do Lex

- Extensões de Expressões regulares

- As expressões regulares no Lex dão a todos estes símbolos um significado especial:
  - \ “ . ^ [ ] * + ? { } | /
  - Seu significado precisa ser desativado se eles forem necessários para representar a si mesmo em uma cadeia. Podemos fazer isso:
    - Colocar o caracter dentro de uma cadeia
      - “* *” casa com * * ou **

- Atividade

- Escreva classes de caracteres para os seguintes conjuntos:
  - As vogais minusculas
  - As dez primeiras letras até j em maiúsculo ou minúsculo
  - Os dígitos em um número hexadecimal(letras maiúsculas para os dígitos acima de 9)
  - Os caracteres que podem aparecer no fim de uma sentença legítima em português(por exemplo, ponto de exclamação).

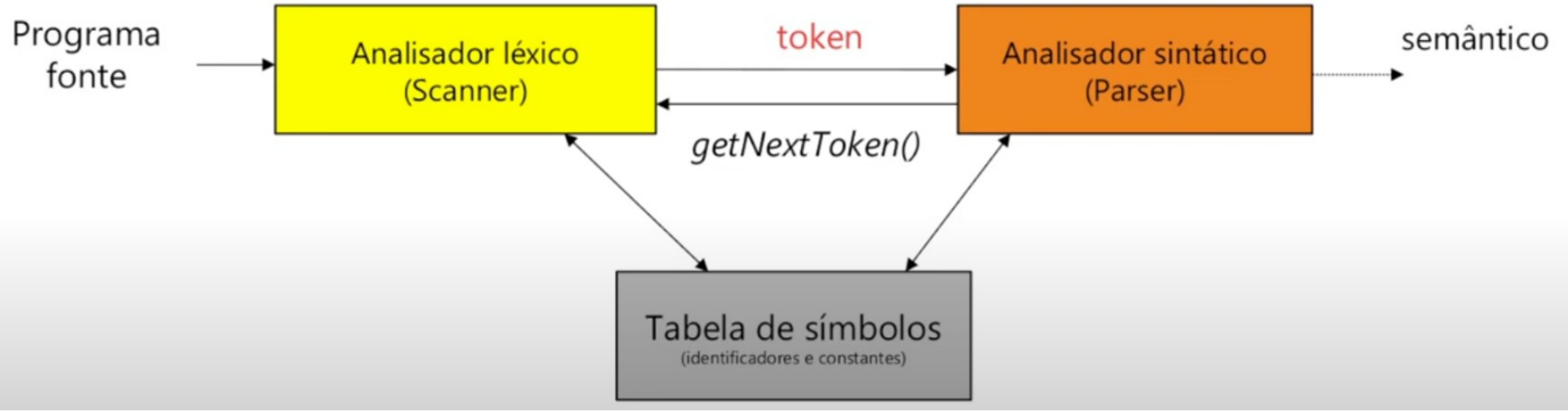
- Atividade

- Escreva uma definição regular para representar os números no formato hexadecimal, iniciando com 0x (ex. 0X27FD01).

- Atividade - resolução

- As vogais minusculas
  - [aeiou]
- As dez primeiras letras até j em maiúsculo ou minúsculo
  - [a-jA-J]
- Os dígitos em um número hexadecimal(letras maiúsculas para os dígitos acima de 9)
  - [0-9A-F]

- Atividade - resolução

- Os caracteres que podem aparecer no fim de uma sentença legítima em português(por exemplo, ponto de exclamação)
  - [.,!?:;]
- Escreva uma definição regular para representar os números no formato hexadecimal, iniciando com 0x (ex. 0X27FD01).
  - Letras -> [a-fA-F]
  - Digitos -> [0-9]
  - Hexa -> 0x(letras|digitos)+

- Diagrama Scanner (Parser)